OpenAI Realtime 마이그레이션 작업 계획

JM Tech One — gpt-realtime-1.5 마이그레이션 범위 + 14일 단계별 계획 + Twilio SIP + Dual-Track 전략

JM Tech One · 2026-05-05 · Internal Strategy Document

JM Tech One — OpenAI Realtime (gpt-realtime-1.5) 마이그레이션 작업 계획

날짜: 2026-05-05 | **대상:** JM Tech One 창업자 (2인 팀) | **현재 상태:** Phase 2-C.B6 완료, 393/393 tests, JM Cafe 라이브 | **목표:** Retell + Gemini 3.1 Flash Lite → OpenAI Realtime (gpt-realtime-1.5) 직접 마이그레이션

---

0. 요약 (Executive Summary)

본 문서는 JM Tech One의 SMB Voice AI 파일럿(JM Cafe Portland)을 현재 스택(Retell + Gemini)에서 OpenAI Realtime API(gpt-realtime-1.5)로 옮기는 데 필요한 모든 의사결정 사항과 작업 계획을 정리합니다. 핵심 결론:

1. **마이그레이션 범위**: "LLM만 교체"가 아닙니다. 전송(Twilio↔Retell) + 오디오 인프라(STT/TTS) + LLM 세 계층 동시 교체이지만, 비즈니스 로직 + DB + 외부 어댑터는 90% 보존됩니다.
2. **소요 시간**: 14일 (2주, 2인 팀 기준) — Phase 0~7.
3. **Twilio TCR과 SIP는 별개**: TCR은 SMS만 영향, OpenAI Realtime 음성 마이그레이션과 무관.
4. **Twilio 연결 방식**: 시작은 **Media Streams WebSocket (방식 A)** 권장 — 디버깅 가시성과 즉시 롤백 가능성이 직접 SIP의 지연 절감 가치보다 큼.
5. **마이그레이션 전략**: **Dual-Track Parallel (듀얼 운영) 필수**. Big-Bang Cutover는 라이브 파일럿에 부적합. 동시 운영 기간 ~7일, 추가 비용 ~$180.

Part 1. 마이그레이션 범위 분석 — LLM만? 전체 재작성?

결론 한 줄

"LLM만 교체"가 아닙니다. 정확히는 **전송(Twilio↔Retell) + 오디오 인프라(STT/TTS) + LLM** 세 계층 동시 교체입니다. 하지만 **비즈니스 로직 + DB + 외부 어댑터는 ~90% 보존**되므로 "전체 프로젝트 재작성"도 아닙니다. 핵심: **`app/api/voice_websocket.py` 한 파일이 새 transport bridge로 대체**되고, 그 외 `app/services/`, `app/skills/`, `app/adapters/`, `app/core/`, DB schema, migrations은 거의 그대로 포팅됩니다.

1.1 현재 아키텍처 ↔ 목표 아키텍처

```
[ 현재 ]
PSTN → Twilio Voice → Retell (audio infra + STT/TTS + agent orchestration)
                        ↓ webhook
                    FastAPI /api/retell/webhook
                        ↓ build prompt + tools
                    Gemini 3.1 Flash Lite (LLM)
                        ↓ function_call
                    bridge_flows.create_order/...
                        ↓ POS/SMS
                    Loyverse / Twilio SMS / SendGrid

[ 목표 ]
PSTN → Twilio Voice (Media Streams 또는 SIP)
                        ↓ WebSocket(audio) 또는 SIP INVITE
                    FastAPI /ws/realtime (or OpenAI direct SIP)
                        ↓ relay audio + tool dispatcher
                    OpenAI Realtime (gpt-realtime-1.5) ← speech-to-speech
                        ↓ function_call event
                    bridge_flows.create_order/...  ← 동일
                        ↓ POS/SMS
                    Loyverse / Twilio SMS / SendGrid  ← 동일
```

1.2 영향 범위 매트릭스 (파일 단위)

🔴 REPLACED (전면 재작성, ~1,500–2,000 LOC)

파일	변경 사유
`app/api/voice_websocket.py` (현재 ~2,200 LOC)	Retell webhook 패턴 → Twilio Media Streams + OpenAI Realtime WebSocket bridge
`app/core/gemini.py`	Gemini engine 폐기
`app/api/retell/*` (있다면)	Retell 의존성 제거
`tests/unit/adapters/test_recital_dedup.py`, `test_modify_clarification.py`, `test_msg_repeat_dedup.py`	RECITAL/MSG DEDUP은 Realtime native VAD가 처리 — 일부 obsolete

🟡 ADAPTED (Tool 스키마 변환, ~200 LOC 변경)

파일	변경 내용
`app/skills/order/order.py`	`function_declarations` (Gemini 형식) → OpenAI `tools` JSON Schema. 로직 동일, 표면만 변환
`app/skills/scheduler/reservation.py`	동일
`app/skills/menu/allergen.py`	동일

파일	변경 내용
`build_system_prompt()`	rule 1–13 그대로, 단 OpenAI Realtime의 `instructions` 필드로 주입 (transport 변경)
8개 tool 정의	형식 변환 — 헬퍼 1회 작성으로 8개 일괄 변환 가능

🟢 PORTED AS-IS (변경 0줄)

영역	파일
Bridge orchestration	`app/services/bridge/flows.py` (create/modify/cancel order, make/modify/cancel reservation 전체)
Order policy engine	`app/services/policy/order_lanes.py` (fire_immediate vs pay_first 결정)
Pay link	`app/services/bridge/pay_link.py`, `pay_link_sms.py`, `pay_link_email.py`, `reservation_email.py`
POS adapter	`app/adapters/loyverse/*` (전체)
SMS adapter	`app/adapters/twilio/sms.py`
DB layer	`app/services/bridge/transactions.py`
RLS / Auth / Config	`app/core/auth.py`, `app/core/config.py`, RLS migrations
DB schema + 모든 migrations	변경 없음
Tool 비즈니스 로직 자체	`render_recall_message`, `is_placeholder_name`, allergen lookup 함수 등 — 모두 동일
Bridge 단위 테스트	`tests/unit/services/` 219건 그대로
Skills 단위 테스트	`tests/unit/skills/` 53건 그대로
Loyverse 어댑터 테스트	`test_loyverse_relay.py` 등 그대로

➕ NEW (신규 추가)

- **`app/api/realtime_bridge.py`** — Twilio Media Streams ↔ OpenAI Realtime WebSocket 양방향 relay (~600 LOC)
- **`app/services/realtime/session.py`** — OpenAI Realtime 세션 관리, function_call 디스패처 (~300 LOC)
- **`app/services/realtime/audio.py`** — 오디오 코덱 변환 (Twilio μ-law 8kHz ↔ OpenAI G.711 통과 — 사실상 transparent)
- **`tests/unit/realtime/*`** — 신규 통합 테스트 (~15–20 케이스)

1.3 정량 요약

항목	수치
전체 코드베이스	~30,000 LOC
교체되는 LOC	~2,500 (8%)
그대로 포팅되는 LOC	~22,000 (73%)
신규 추가 LOC	~1,000 (3%)
변경 없는 LOC (테스트 + adapter + DB)	~5,500 (16%)
393/393 테스트 중 그대로 통과 예상	**~330개 (84%)**
재작성 필요 테스트	**~63개 (16%, 주로 voice integration)**

Part 2. 14일 단계별 마이그레이션 계획

Phase 0 — 사전 준비 (Day 0, 0.5일)

목표: 차단 요소 제거, 환경 설정.

- [ ] OpenAI 계정 Tier 검증 (Realtime API 가용 Tier ≥ 1)
- [ ] OpenAI API 키 발급 + `.env` 에 `OPENAI_API_KEY` 추가
- [ ] `gpt-realtime-1.5` 모델 가용성 확인 (region 선택)
- [ ] OpenAI Trust Center에서 ZDR (Zero Data Retention) 신청 (선택)
- [ ] Python SDK 추가: `openai>=1.50.0` (Realtime 클라이언트 포함)
- [ ] `feature/openai-realtime-migration` 브랜치 생성
- [ ] `git tag pre-realtime-migration` 으로 롤백 포인트 표시

롤백 절차: 브랜치 폐기로 즉시 복구.

Phase 1 — POC: Echo Bot (Day 1–2, 2일)

목표: WebSocket 양방향 오디오 통과 확인. 가장 싸게 fail-fast.

- [ ] `gpt-realtime-mini` 사용 (Phase 1만, POC 비용 ↓)
- [ ] 단순 Echo: 사용자 발화 → OpenAI Realtime → 그대로 echo TTS
- [ ] 테스트 도구: OpenAI Playground 또는 `wscat` 으로 WebSocket 직접 연결
- [ ] 검증: `session.update` → `input_audio_buffer.append` → `response.audio.delta` 라이프사이클
- [ ] 측정: 첫 청크 TTFT (목표 <300ms)

산출: `scripts/poc_realtime_echo.py` (참고용, 폐기 가능)

Phase 2 — Twilio Media Streams 브릿지 (Day 3–4, 2일)

목표: 실제 PSTN 통화 ↔ OpenAI Realtime 연결.

- [ ] Twilio Voice 번호 신규 또는 기존 활용
- [ ] TwiML 설정: 인바운드 콜 → `<Connect><Stream url="wss://api.jmtechone.com/ws/realtime"/></Connect>`
- [ ] FastAPI WebSocket 엔드포인트 `/ws/realtime` 구현
- Twilio Media Streams 프로토콜 (start/media/stop frames) 디코드
- μ-law 8kHz audio → OpenAI Realtime `input_audio_buffer.append` (G.711 μ-law 직접 통과 — 변환 불필요)
- OpenAI Realtime `response.audio.delta` (G.711 μ-law) → Twilio Media Streams `media` frame
- [ ] Caller-ID 추출: Twilio `start` event의 `customParameters` 또는 `accountSid` + `callSid` 조회 → 매장 식별
- [ ] 매장 lookup → tenant_id 설정
- [ ] **목표 TTFT 측정 (PSTN end-to-end <500ms)**

산출: `app/api/realtime_bridge.py`, `app/services/realtime/session.py`, `app/services/realtime/audio.py`

롤백: Twilio TwiML을 Retell URL로 다시 가리키면 즉시 복구.

Phase 3 — 시스템 프롬프트 + 8개 Tool 포팅 (Day 5–6, 2일)

목표: Gemini가 하던 일을 Realtime이 하도록 함수 호출 안정화.

- [ ] `build_system_prompt()` 결과를 `session.instructions`로 주입
- rule 1–13 그대로, 단 "READ MESSAGE VERBATIM" 같은 Retell-specific 문구는 Realtime context에 맞게 미세 조정
- [ ] 8개 voice tool을 OpenAI `tools` 형식으로 변환:

```
# 변환 헬퍼 (1회 작성)
def gemini_to_openai_tool(gemini_def):
    decl = gemini_def["function_declarations"][0]
    return {
        "type": "function",
        "name": decl["name"],
        "description": decl["description"],
        "parameters": decl["parameters"],
    }

# 적용
TOOLS = [gemini_to_openai_tool(t) for t in [
    ORDER_TOOL_DEF, MODIFY_ORDER_TOOL_DEF, CANCEL_ORDER_TOOL_DEF,
    RESERVATION_TOOL_DEF, MODIFY_RESERVATION_TOOL_DEF, CANCEL_RESERVATION_TOOL_DEF,
    ALLERGEN_LOOKUP_TOOL_DEF, RECALL_ORDER_TOOL_DEF,
]]
```

- [ ] Function call 이벤트 디스패처: `response.function_call_arguments.done` → `bridge_flows.create_order(...)` (기존 함수 그대로 호출)
- [ ] Tool 결과 반환: `conversation.item.create` (function_call_output) → `response.create`
- [ ] **Async tool exec**: Loyverse 호출 2–5초 → asyncio Task로 처리, 동안 placeholder TTS ("Just a moment...")
- [ ] **Semantic VAD `eagerness=low`**: 예약 readback 등 긴 어시스턴트 턴 보호

산출: 8개 tool 모두 동작, function_call 정확도 검증.

Phase 4 — 결제 링크 통합 (Day 7, 1일)

목표: pay link SMS/Email 자동 발송 보존.

- [ ] `create_order` success 후 `pay_link_sms.send_pay_link()` + `pay_link_email.send_pay_link_email()` async fire — **기존 코드 그대로 호출**, transport만 변경
- [ ] Closing-summary line (Proposal I) 동등 구현: Realtime 통화 종료 이벤트(`session.terminated` or WebSocket close)에서 발화
- [ ] 예약 이메일 deferred fire: 동일 패턴

산출: pay link, reservation email 모두 정상 발송.

Phase 5 — 회귀 테스트 + 라이브 시뮬레이션 (Day 8, 1일)

목표: 393/393 → 새 구조에서도 안정.

- [ ] `tests/unit/services/`, `tests/unit/skills/`, `tests/unit/adapters/test_loyverse_relay.py` 등 **그대로 통과 확인** (~330건)
- [ ] 신규 `tests/unit/realtime/test_session.py`, `test_tool_dispatch.py` 작성 (~15건)
- [ ] Voice integration 테스트 일부 재작성 (RECITAL DEDUP은 obsolete, 기능 검증으로 대체)
- [ ] **모델은 `gpt-realtime-1.5` 로 전환** (POC mini → production 1.5)
- [ ] 본인이 직접 시나리오 10통 실행 (주문/수정/취소/예약/알러지/EpiPen/recall_order/다국어 switching)

Acceptance Gate:

- TTFT ≤ 500ms (PSTN end-to-end)
- Function call accuracy ≥ 95% (단일 의도 기준)
- B5 Tier-3 EpiPen 핸드오프 100% 동작
- 모든 시나리오 기존 동작 등가 또는 개선

Phase 6 — Canary 5–10% (Day 9–11, 3일)

목표: 라이브 트래픽에서 검증.

- [ ] Twilio Voice 번호 두 개 운영: 기존 Retell 번호 + 신규 Realtime 번호
- [ ] 매장 안내문구로 "신규 시스템 테스트 중" 통화는 신규 번호로 유도 (또는 caller-id 기반 라우팅 5–10%)
- [ ] Sentry / 자체 모니터로 실시간 추적: TTFT, function_call 실패율, 통화 완료율

- [ ] 일 1회 통화 샘플 5건 수동 리뷰
- [ ] **롤백 트리거**: function_call 실패율 >5% 또는 TTFT >800ms 5분 sustained → 즉시 Retell 번호로 트래픽 전체 회수

Phase 7 — 풀 롤아웃 + Retell 제거 (Day 12–14, 3일)

목표: Realtime 100%, Retell 코드 정리.

- [ ] 매장 메인 번호 TwiML을 Realtime endpoint로 영구 전환
- [ ] 7일 모니터링 후 회귀 없으면 Retell 계약 해지 사전 통지
- [ ] `app/api/voice_websocket.py` (Retell handler) 삭제
- [ ] `app/core/gemini.py` 삭제
- [ ] `requirements.txt` 에서 Retell SDK + Gemini SDK 제거
- [ ] Obsolete 테스트 정리 (RECITAL DEDUP 등)
- [ ] CLAUDE.md / MEMORY 업데이트

14일 일정 한눈에

```
Day 0:    Phase 0 (env/keys/branch)
Day 1-2:  Phase 1 (POC echo)
Day 3-4:  Phase 2 (Twilio Media Streams bridge)
Day 5-6:  Phase 3 (system prompt + 8 tools)
Day 7:    Phase 4 (pay link)
Day 8:    Phase 5 (regression)
Day 9-11: Phase 6 (canary 5-10%)
Day 12-14: Phase 7 (full rollout + Retell removal)
```

Part 3. Twilio TCR + SIP — 자세한 분석

3.1 용어 명확화 — TCR과 SIP는 별개

용어	무엇	음성에 영향?
TCR (The Campaign Registry)	미국 10DLC SMS 캠페인 등록 — A2P SMS 발송 인증	❌ 음성 통화에 **무관**
A2P 10DLC	TCR이 관리. SMS pay link 발송 시에만 영향	❌ 음성 무관
SIP Trunking	PSTN ↔ 인터넷 음성 연결 프로토콜	✅ 음성의 핵심
Twilio Elastic SIP Trunk	Twilio가 제공하는 SIP 트렁크 상품	✅
Programmable Voice	Twilio 일반 음성 번호 (TwiML 사용)	✅

> ***결론**: 현재 대기 중인 **Twilio TCR 승인은 SMS pay link 도달률만 영향**. OpenAI Realtime SIP 마이그레이션과 **완전히 별개**입니다. 음성 통화는 TCR 승인 없이도 즉시 가능.*

3.2 OpenAI Realtime ↔ Twilio 두 가지 연결 방식

방식 A: Twilio Media Streams (WebSocket) — 권장 시작 옵션

```
PSTN → Twilio Voice → TwiML <Stream> → Your Backend WebSocket → OpenAI Realtime
WebSocket
                                        (audio relay + tool dispatcher)
```

Twilio 측 작업:

1. Twilio 콘솔 → Phone Numbers → 매장 번호 선택
2. Voice Configuration → Webhook URL: 다음 TwiML을 반환하는 endpoint:

```
<Response>
  <Connect>
    <Stream url="wss://api.jmtechone.com/ws/realtime">
      <Parameter name="store_id" value="{매장 ID}"/>
    </Stream>
  </Connect>
</Response>
```

3. 끝. Twilio 측에서 추가 승인/등록 절차 **없음**.

OpenAI 측 작업: 없음. API 키만 있으면 클라이언트로 WebSocket 연결.

장점:

- 우리 백엔드가 모든 audio 트래픽 통과 → 완전 가시성 (디버깅, 로깅, 오디오 녹음, 안전 가드)
- mid-stream에 placeholder TTS 삽입 가능 ("잠시만요…" 동안 Loyverse 호출)
- function_call 이벤트를 backend가 직접 받음 — 기존 dispatcher 패턴 그대로
- Twilio 신규 가입자도 즉시 사용 가능, 추가 비용 없음

단점:

- 우리 서버를 한 번 더 거침 → 50–100ms 추가 지연 (PSTN 끝–끝 ~400–500ms)
- 우리 서버가 audio relay 부하를 짊어짐 (WebSocket 연결 수 = 동시 통화 수)

방식 B: OpenAI 직접 SIP — 고급 옵션, Phase 8+ 검토

```
PSTN → Twilio Elastic SIP Trunk → SIP INVITE → sip:proj_<ID>@sip.api.openai.com
                                                (OpenAI가 직접 audio 처리)

                                                Function calls → HTTPS webhook → Your
Backend
```

Twilio 측 작업 (방식 A보다 복잡):

1. Twilio Console → **Elastic SIP Trunking** 메뉴 (Programmable Voice와 별개 제품)
2. 신규 SIP Trunk 생성 (예: `jm-realtime-trunk`)
3. **Termination URI**: `sip:proj_abc123@sip.api.openai.com` 입력
4. **Origination**: 인바운드 매장 번호 → 이 트렁크로 라우팅
5. IP Access Control List: OpenAI SIP 서버 IP 범위 허용
6. **인증**: OpenAI 측 프로젝트 API 키 기반 SIP credentials

OpenAI 측 작업:

- OpenAI 대시보드 → Project Settings → Realtime SIP 활성화
- SIP credentials 발급 + 허용 매장(번호) 등록
- 시스템 프롬프트 + tools를 Project default 또는 session 단위로 설정

장점:

- 가장 낮은 지연 (Twilio→OpenAI direct, 우리 서버 미경유 → ~50–100ms 단축)
- 우리 서버 부하 ↓ (audio relay 없음)
- 통화 동시성 한도가 우리 서버 capacity가 아닌 Twilio + OpenAI 한도로 결정

단점:

- audio가 우리 서버를 안 지나감 → **녹음/디버깅/안전 가드 곤란**
- function_call이 out-of-band — HTTPS webhook으로만 받음 (실시간 audio context와 동기화 더 어려움)
- 인터럽션, mid-call placeholder TTS, 동적 시스템 프롬프트 변경이 제한적
- SIP 설정 디버깅 난이도 ↑ (네트워크 레벨 troubleshooting)
- Twilio Elastic SIP Trunk는 **별도 제품** — 분당 요금이 Programmable Voice와 다름

3.3 권장 진로

> ***Phase 1–7 (이번 14일 마이그레이션)은 방식 A (Media Streams WebSocket)** 사용.*
> ***방식 B (직접 SIP)는 5매장 확장 후 Phase 8+ 별도 트랙**으로 평가.*

이유:

1. **Day 1 자체 디버깅 가능** — audio가 우리 서버를 통과해야 dev 단계 문제 추적 빠름
2. **TTFT 200ms 차이는 Phase 1에선 무관** — 현재 Retell+Gemini가 800–1300ms, Realtime+Media Streams 400–500ms로 가도 **2배 개선** 이미 충분

3. **Twilio TCR 승인 무관** — 즉시 출발 가능
4. **롤백 단순** — TwiML URL을 Retell webhook으로 다시 가리키면 즉시 복구
5. **방식 B로 가더라도 backend tool dispatcher 코드는 재사용** — 매몰 비용 거의 없음

3.4 본인이 직접 Twilio에서 할 작업 (방식 A)

단계	위치	시간
1. 매장 번호 확인	Twilio Console → Phone Numbers	5분
2. Voice Configuration → A Call Comes In: 우리 백엔드의 TwiML endpoint URL 입력	Number 상세 페이지	5분
3. TwiML endpoint(`/twilio/voice/inbound`)가 `<Connect><Stream>` XML 반환하도록 구현	우리 백엔드 코드	20분
4. ngrok 또는 prod 도메인의 WebSocket endpoint(`/ws/realtime`) 가용 확인	우리 백엔드	—
5. 테스트 콜 1통 → Media Streams audio 흐름 확인	휴대폰	2분

> ***추가 Twilio 승인/등록 절차 없음**. 음성 번호만 있으면 즉시 작동.*

3.5 비용 비교 (5매장 × 50통화/일 × 5분 × 30일 = 37,500분/월 기준)

항목	방식 A (Media Streams)	방식 B (직접 SIP)
Twilio 인바운드 (Programmable Voice)	$0.0085/분 × 37,500 = **$319/월**	—
Twilio Elastic SIP Trunk 인바운드	—	$0.004/분 × 37,500 = **$150/월**
Twilio Media Streams	포함	해당 없음
OpenAI gpt-realtime-1.5	~$0.031/분 × 37,500 = **$1,160/월**	동일
우리 서버 audio relay 부하	EC2 c5.large 추가 약 ~$60/월	0
합계	**~$1,540/월**	**~$1,310/월** (~15% 절감)

> *5매장 확장 시점에서 **월 $230 절감** — 방식 B 마이그레이션 검토 가치 있음. 하지만 Phase 1–7 동안은 방식 A의 **디버깅 가시성 + 즉시 롤백 가능**이 절감액보다 가치 큼.*

Part 4. 마이그레이션 전략 — Dual-Track vs Big-Bang vs Strangler

4.1 핵심 질문

> *현재 아키텍처를 그대로 유지하면서 목표 아키텍처를 만들어 듀얼로 운영 후 이전할 것인가, 아니면 처음부터 바로 신규 아키텍처로 갈 것인가?*

결론: Dual-Track Parallel (듀얼 운영) 필수. JM Cafe가 이미 라이브 파일럿이므로 Big-Bang Cutover는 영업 중단 위험을 직접 감수해야 합니다. 단, 듀얼 운영의 정확한 의미는 "두 코드베이스 유지"가 아니라 **"같은 코드베이스 + 두 transport adapter 공존"**입니다.

4.2 세 가지 전략 비교

전략	설명	적합성
A. Big-Bang Cutover	기존 stack 정지 → 신규 stack 빌드 → 모든 트래픽 한 번에 전환	❌
B. Dual-Track Parallel	같은 codebase에 두 transport 공존, 트래픽 점진 전환 (0→5→25→50→100%)	✅ **권장**
C. Strangler Fig	기능 단위로 점진 이전 (예: 알러지만 Realtime, 나머지 Retell)	❌

A. Big-Bang Cutover — 반대

작동 방식:
- D-day 정해서 한 번에 모든 트래픽을 신규 stack으로 전환

장점:
- 코드 단순 (이중 유지 부담 없음)
- 빠른 정리 (Retell/Gemini 코드 즉시 삭제)

단점 (이번 케이스에서 치명적):
- 회귀 발생 시 JM Cafe 영업 즉시 중단 — 분 단위 매출 손실
- 롤백 = 코드 revert + 재배포 + Twilio TwiML 변경 + DB 상태 점검 = 30+ 분
- 라이브 파일럿 + 2인 팀이라 24/7 대응 불가
- Acceptance gate 통과해도 라이브 트래픽 패턴은 dev/staging과 다름 (예: 노이즈 환경, 사투리, 동시 통화)

평가: ❌ 라이브 운영 중인 파일럿에 부적합

B. Dual-Track Parallel — 권장

작동 방식:
- 같은 main 브랜치, 두 transport adapter 공존
- `app/api/voice_websocket.py` (Retell, 기존) — 그대로 유지
- `app/api/realtime_bridge.py` (Realtime, 신규) — 추가
- 두 adapter 모두 동일한 `bridge_flows.create_order(...)` 등 비즈니스 로직 호출
- DB는 단일 source of truth — 두 stack에서 들어온 주문이 같은 테이블에 저장
- Twilio 측에서 라우팅:

- 매장 메인 번호 → Retell (default, 95%)
- 신규 테스트 번호 → Realtime (canary, 5%)
- 라이브 검증 후 점진 전환: 5% → 25% → 50% → 100%

장점:
- 즉시 롤백 (TwiML URL flip만으로 30초 내 100% Retell로 회귀)
- JM Cafe 영업 중단 위험 0
- 같은 DB → 양 stack의 데이터가 통합되어 KPI 비교 가능 (TTFT, function_call 정확도, 통화 완료율, 고객 만족도)
- 같은 codebase → 두 갈래 maintain 부담 최소 (transport layer만 다름)
- Phase 6의 5–10% canary와 Phase 7의 점진 전환에 자연스럽게 연결

단점:
- 7일 동안 Retell + OpenAI 비용 동시 부담 (~$180 추가)
- transport 코드 두 개 동시 디버깅 필요 (단, Phase 7에서 Retell 제거)

평가: ✅ 라이브 파일럿에 표준 패턴

C. Strangler Fig — 부적합

작동 방식:
- 기능 단위로 점진 이전 (예: 알러지 Q&A만 먼저 Realtime, 나머지는 Retell)
- 통화 mid-call에 stack 전환

문제:
- **음성 세션은 분할 불가** — 통화 중간에 audio stream을 다른 LLM으로 hop 시키면 latency + context 손실 폭발
- function_call 라우팅 복잡도 폭증
- 사용자 경험 저하 (통화 중 목소리 변경)

평가: ❌ 음성 도메인에 부적합 (텍스트 챗봇이라면 가능)

4.3 Dual-Track 구체적 구현

4.3.1 코드 레이아웃

```
backend/app/
├── api/
│   ├── voice_websocket.py       # Retell handler (기존, 그대로 유지)
│   └── realtime_bridge.py       # OpenAI Realtime handler (신규 추가)
├── services/
│   ├── bridge/
│   │   └── flows.py             # ★ 단일 source of truth (양 adapter 호출)
│   ├── realtime/                # 신규
│   │   ├── session.py
│   │   ├── audio.py
│   │   └── tool_dispatcher.py
│   └── policy/                  # 그대로
├── skills/                      # 그대로 (스키마 헬퍼만 추가)
├── adapters/                    # 그대로
└── core/                        # gemini.py만 Phase 7에서 제거
```

4.3.2 트래픽 라우팅 (Twilio 레벨)

단계	**Retell (기존 번호)**	**Realtime (신규 번호)**	기간
Phase 6.1 (Day 9)	95%	5%	1일
Phase 6.2 (Day 10)	75%	25%	1일
Phase 6.3 (Day 11)	50%	50%	1일
Phase 7.1 (Day 12)	25%	75%	1일
Phase 7.2 (Day 13)	0%	100%	1일
Phase 7.3 (Day 14)	(Retell 코드 제거)	100%	1일

라우팅 방식 옵션:

- **옵션 1 (단순)**: 두 개의 번호 운영 — 안내문에 "신규 시스템 테스트: 503-XXX-NEW" 안내
- **옵션 2 (정밀)**: 단일 번호 + caller-id 해시로 5%/25%/50%/100% 분기 — Twilio Function 또는 우리 백엔드의 dispatcher
- **옵션 3 (시간 기반)**: 영업 시간대 별로 분기 (오전 = Retell, 오후 = Realtime)

권장: 옵션 1로 시작 → Phase 6.2부터 옵션 2

4.3.3 회귀 트리거 + 롤백 절차

메트릭	임계값	트리거 시 액션
Function call 실패율	>5% sustained 5분	Twilio TwiML URL을 Retell endpoint로 즉시 flip (30초 내)
TTFT 95p	>800ms sustained 5분	동일
통화 완료율	<90% sustained 1시간	동일

메트릭	임계값	트리거 시 액션
EpiPen 핸드오프 실패	1건이라도	즉시 + 사후 분석
사용자 컴플레인	1건	Realtime 트래픽 percentage 50% 감소 후 분석

롤백 액션 자체:

1. Twilio Console → Phone Number → Voice Configuration
2. Webhook URL을 `/api/retell/webhook` (기존)으로 변경
3. Save
4. 다음 통화부터 Retell로 라우팅 (in-flight 통화는 그대로 완료)

총 소요: **30–60초**.

4.4 동시 운영 비용 (1매장 7일 기준)

항목	**Retell (기존)**	**OpenAI Realtime (신규)**	합계
Phase 6 (Day 9–11): 평균 25% Realtime, 75% Retell	$0.07/분 × 75% × 1,750분 = **$92**	$0.031/분 × 25% × 1,750분 = **$14**	$106
Phase 7.1–7.2 (Day 12–13): 평균 50% Realtime	$0.07/분 × 50% × 500분 = **$18**	$0.031/분 × 50% × 500분 = **$8**	$26
7일 합계 동시 운영 추가 부담	—	—	**~$132**
Twilio Voice (양쪽 공통)	$0.0085/분 × 2,250분 = **$19**	(포함)	$19
7일 총 추가 비용 (vs Big-Bang)	—	—	**~$151**

> ***$151의 안전 보험**으로 라이브 파일럿 영업 중단 위험을 제거. Big-Bang 1회 1시간 영업 중단의 기회비용(고객 신뢰 손실 + 매출)이 훨씬 큼.*

4.5 듀얼 운영의 위험 매트릭스

위험	가능성	영향	대응
두 stack의 DB 동시 접근으로 race condition	중	중	RLS + idempotent guard 이미 적용됨 — 추가 변경 불필요
Retell이 dev 중 우연히 신규 endpoint 호출	저	저	URL이 다르므로 충돌 불가
같은 고객이 두 번호 모두 호출	저	저	caller-id 기반 dedup 이미 작동 — 양 stack 공유

위험	가능성	영향	대응
Phase 6 중 OpenAI rate limit	중	중	gpt-realtime-mini로 즉시 fallback (Phase 6 acceptance gate에 포함)
두 transport 코드 모두 디버깅 부담	고	저	Phase 6는 7일만 — 일시적

4.6 권장 실행 순서

```
Day 0-8:  Dual-Track 준비
  ├─ feature 브랜치에서 realtime_bridge 추가 (voice_websocket 미접촉)
  ├─ 회귀 테스트 통과 후 main 머지 (라이브 트래픽은 여전히 Retell)
  └─ Acceptance gate 통과

Day 9-11: Dual-Track 라이브 (Phase 6)
  ├─ 신규 번호로 5% → 25% → 50% canary
  ├─ KPI 추적 (TTFT, function_call, 완료율)
  └─ 회귀 트리거 시 즉시 롤백

Day 12-13: 점진 전환 (Phase 7.1-7.2)
  ├─ 75% → 100% Realtime
  └─ Retell은 비상 회귀용 standby

Day 14:    풀 cutover (Phase 7.3)
  ├─ Retell 코드 제거
  ├─ 7일 추가 모니터링 후 Retell 계약 해지
  └─ 듀얼 운영 종료
```

다음 액션 (요약)

1. **Phase 0 시작 승인**: feature 브랜치 + OpenAI API 키 + requirements.txt
2. **방식 A (Twilio Media Streams)**로 시작 — 디버깅 가시성 우선
3. **Dual-Track Parallel** 채택 — 7일간 ~$150 추가 비용 감수, 영업 중단 위험 0
4. **Big-Bang은 절대 금지** — 라이브 파일럿에 부적합
5. **방식 B (직접 SIP)는 Phase 8+ 5매장 확장 시 별도 평가** — 월 $230 절감 잠재력은 인지하나 현재는 가시성이 더 중요

총 소요: 14일 / 추가 비용: ~$150 / 위험: 즉시 롤백 가능 / 결과: TTFT 800–1300ms → 400–500ms (~50% 개선) + 다국어 mid-call switching + tool 정확도 +7%